Chat System Daemon - Continuous Monitoring & Auto-Recovery

Overview

The chat system now uses a **continuously running daemon** that monitors all services and automatically maintains their health. Unlike traditional systemd oneshot services that just start containers and exit, this daemon stays active and provides intelligent health monitoring and recovery.

Features

Continuous Monitoring

- **Check Interval:** 30 seconds
- **Monitored Services:** Redis, NATS, chat-node-1, chat-node-2, chat-node-3, chat-lb
- **Health Checks:**
  - Container running status
  - Podman health check status
  - HTTP health endpoint validation (for services with HTTP endpoints)

Automatic Recovery

1. **Restart:** If a container is unhealthy or stopped, daemon attempts restart
2. **Recreate:** If restart fails, daemon removes and recreates the container
3. **Cooldown:** 60-second cooldown between restarts to prevent restart loops
4. **Logging:** All actions logged to `/var/log/chat-system-daemon.log`

Intelligent Failure Handling

- Detects stopped containers
- Detects unhealthy containers (via health checks)
- Detects HTTP endpoint failures
- Prevents restart storms with cooldown periods
- Recreates containers with correct configuration if restart fails

PROF

Architecture

```
systemd (chat-system.service)
    ↓
ExecStartPre: chat-system.sh start (start all containers)
    ↓
ExecStart: chat-system-daemon.sh (continuous monitoring loop)
    ↓ (every 30s)
    ├─ Check Redis
    ├─ Check NATS
    ├─ Check chat-node-1
```

```
    ├─ Check chat-node-2
    ├─ Check chat-node-3
    └─ Check chat-lb
         ↓ (if unhealthy)
         ├─ Restart container
         └─ Recreate if restart fails
    ↓
ExecStopPost: chat-system.sh stop (cleanup on daemon exit)
```

Setup

Initial Configuration

```
sudo ./chat-system.sh setup-autostart
```

This will:

1. Ask for configuration (nodes, cluster mode, load balancer, etc.)
2. Create `/etc/systemd/system/chat-system.service`
3. Enable the service for autostart on boot
4. Optionally start the daemon immediately

Manual Service File Creation

If you need to manually configure the service:

```
[Unit]
Description=Distributed Chat System with Continuous Health Monitoring
After=network-online.target
Wants=network-online.target

[Service]
Type=simple
User=root
Group=root
WorkingDirectory=/path/to/chat-system-test
Environment="PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbi
n:/bin"
Environment="CLUSTER_CONSUL_URL=http://192.168.100.52:8500"

# Start services first
ExecStartPre=/path/to/chat-system.sh start --nodes 3 --cluster --
cluster-consul http://192.168.100.52:8500 --mode full --with-lb

# Run monitoring daemon
ExecStart=/path/to/chat-system-daemon.sh

# Stop services on daemon exit
```

PROF

```
ExecStopPost=/path/to/chat-system.sh stop

# Restart daemon if it crashes
Restart=on-failure
RestartSec=10

StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Usage

Start Daemon

```
sudo systemctl start chat-system
```

This will:

1. Start all containers (via ExecStartPre)
2. Begin continuous monitoring loop
3. Log daemon startup to `/var/log/chat-system-daemon.log`

Stop Daemon

```
sudo systemctl stop chat-system
```

This will:

PROF

1. Send SIGTERM to daemon (graceful shutdown)
2. Run cleanup script (via ExecStopPost)
3. Stop and remove all containers

Check Status

```
# Quick status
./chat-system.sh status

# Detailed systemd status
sudo systemctl status chat-system

# View daemon logs
sudo tail -f /var/log/chat-system-daemon.log
```

```
# View systemd journal
sudo journalctl -u chat-system -f
```

Enable Autostart on Boot

```
sudo systemctl enable chat-system
```

Disable Autostart

```
sudo systemctl disable chat-system
```

Monitoring

Daemon Logs

The daemon logs all actions to `/var/log/chat-system-daemon.log`:

```
# View last 50 lines
sudo tail -50 /var/log/chat-system-daemon.log

# Follow logs in real-time
sudo tail -f /var/log/chat-system-daemon.log

# Search for specific container
sudo grep "chat-node-2" /var/log/chat-system-daemon.log
```

Log Format:

PROF

```
[2026-01-12 14:17:20] Chat System Daemon Started
[2026-01-12 14:17:20] Check Interval: 30s
[2026-01-12 14:17:20] Restart Cooldown: 60s
[2026-01-12 14:18:57] ERROR: Container chat-node-2 not running (status:
exited)
[2026-01-12 14:18:57] Service chat-node-2 in cooldown (32s/60s)
[2026-01-12 14:19:29] Restarting container: chat-node-2
[2026-01-12 14:19:29] SUCCESS: Container chat-node-2 restarted
[2026-01-12 14:20:00] Heartbeat: Check #10 - All services monitored
```

Systemd Logs

View systemd journal for daemon activity:

```
# Recent logs
sudo journalctl -u chat-system -n 50

# Follow logs
sudo journalctl -u chat-system -f

# Logs since today
sudo journalctl -u chat-system --since today
```

Container Status

Check container health:

```
# All containers
sudo podman ps --format "table {{.Names}}\t{{.Status}}"

# Specific container health
sudo podman inspect chat-node-1 --format '{{.State.Health.Status}}'

# Container restart count
sudo podman inspect chat-node-1 --format '{{.State.RestartCount}}'
```

Behavior Examples

Example 1: Container Stopped

Scenario: chat-node-2 is manually stopped

```
sudo podman stop chat-node-2
```

PROF

Daemon Response (within 30s):

```
[2026-01-12 14:18:57] ERROR: Container chat-node-2 not running (status:
exited)
[2026-01-12 14:18:57] Restarting container: chat-node-2
[2026-01-12 14:18:57] SUCCESS: Container chat-node-2 restarted
```

Result: Container is automatically restarted

Example 2: Container Unhealthy

Scenario: Container's health check fails (e.g., HTTP endpoint not responding)

Daemon Response:

```
[2026-01-12 14:20:15] ERROR: Container chat-node-1 is unhealthy
[2026-01-12 14:20:15] Restarting container: chat-node-1
[2026-01-12 14:20:15] SUCCESS: Container chat-node-1 restarted
```

Result: Container is restarted to restore health

Example 3: Container Crashed

Scenario: Container process crashes inside container

Daemon Response:

```
[2026-01-12 14:22:30] ERROR: HTTP health check failed for chat-node-3 on
port 3003
[2026-01-12 14:22:30] Restarting container: chat-node-3
[2026-01-12 14:22:31] SUCCESS: Container chat-node-3 restarted
```

Result: Container is detected and restarted

Example 4: Restart Failure → Recreate

Scenario: Container restart fails (corrupted state)

Daemon Response:

```
[2026-01-12 14:25:00] ERROR: Container redis is unhealthy
[2026-01-12 14:25:00] Restarting container: redis
[2026-01-12 14:25:00] ERROR: Failed to restart redis
[2026-01-12 14:25:00] Recreating container: redis (restart failed)
[2026-01-12 14:25:01] Recreating Redis...
[2026-01-12 14:25:02] SUCCESS: Redis recreated
```

Result: Container is removed and recreated from scratch

Example 5: Cooldown Prevention

Scenario: Same container fails multiple times rapidly

Daemon Response:

```
```

```
[2026-01-12 14:30:00] ERROR: Container chat-lb not running (status:
exited)
[2026-01-12 14:30:00] Restarting container: chat-lb
[2026-01-12 14:30:00] SUCCESS: Container chat-lb restarted
[2026-01-12 14:30:30] ERROR: Container chat-lb not running (status:
exited)
[2026-01-12 14:30:30] Service chat-lb in cooldown (30s/60s)
```

Result: Daemon waits 60 seconds before attempting another restart to prevent restart loops

---

Configuration

Adjusting Check Interval

Edit `/path/to/chat-system-daemon.sh`:

```
CHECK_INTERVAL=30  # Change to desired seconds (default: 30)
```

Then reload daemon:

```
sudo systemctl daemon-reload
sudo systemctl restart chat-system
```

Adjusting Cooldown Period

Edit `/path/to/chat-system-daemon.sh`:

PROF

```
RESTART_COOLDOWN=60  # Change to desired seconds (default: 60)
```

Adding More Services

To monitor additional containers, edit the main loop in `chat-system-daemon.sh`:

```
# Add to main() function
monitor_service "your-container-name" "port-number"
```

Troubleshooting

Daemon Won't Start

Check service file:

```
sudo systemctl cat chat-system
```

Check for errors:

```
sudo systemctl status chat-system
sudo journalctl -u chat-system -n 50
```

Verify script exists and is executable:

```
ls -l /path/to/chat-system-daemon.sh
chmod +x /path/to/chat-system-daemon.sh
```

Daemon Keeps Restarting Services

Check daemon logs for errors:

```
sudo tail -100 /var/log/chat-system-daemon.log
```

Common causes:

- Containers failing health checks
- HTTP endpoints not responding
- Port conflicts
- Resource exhaustion (CPU/memory)

PROF

Increase cooldown period temporarily:

```
# Edit daemon script
RESTART_COOLDOWN=300  # 5 minutes
```

Containers Not Being Recreated

Check recreate functions:

```
# Test Redis recreation manually
sudo podman stop redis
sudo podman rm redis
# Wait for daemon to recreate (within 30s + 60s cooldown)
```

Verify image exists:

```
sudo podman images | grep chat-node
```

Daemon Logs Growing Too Large

Logs auto-rotate to 1000 lines, but you can manually clean:

```
# Truncate log file
sudo truncate -s 0 /var/log/chat-system-daemon.log

# Or archive old logs
sudo mv /var/log/chat-system-daemon.log /var/log/chat-system-
daemon.log.old
```

Comparison: Old vs New

Old System (Type=oneshot with timer)

- ✗ Started containers and exited
- ✗ Separate timer checked every 2 minutes
- ✗ Limited restart logic
- ✗ No recreation on restart failure
- ✗ No cooldown periods

New System (Type=simple with daemon)

- ✅ Continuously running daemon
- ✅ Checks every 30 seconds
- ✅ Intelligent restart with fallback to recreate
- ✅ Prevents restart loops with cooldowns
- ✅ Detailed logging of all actions
- ✅ Heartbeat logging every 5 minutes

PROF

Best Practices

1. **Monitor daemon logs regularly:**

   ```
   sudo tail -f /var/log/chat-system-daemon.log
   ```

2. **Check daemon status after boot:**

```
./chat-system.sh status
```

3. **Review logs after incidents:**

```
sudo grep "ERROR" /var/log/chat-system-daemon.log
```

4. **Test recovery by stopping a container:**

```
sudo podman stop chat-node-1
# Wait 30-60s, verify it restarts
```

5. **Use systemd status for quick health check:**

```
sudo systemctl is-active chat-system
```

Performance Impact

- **CPU:** Minimal (<0.1% average)
- **Memory:** ~6-10 MB for daemon process
- **Disk I/O:** Negligible (log file, rotated at 1000 lines)
- **Network:** None (local podman socket communication)

Security Considerations

- Daemon runs as **root** (required for podman access)
- Log file readable by **root only** (`/var/log/chat-system-daemon.log`)
- No network exposure (local monitoring only)
- systemd manages daemon lifecycle (automatic restart on crash)

PROF

Migration from Old System

If upgrading from the old timer-based system:

1. **Stop old services:**

```
sudo systemctl stop chat-system chat-system-monitor.timer
sudo systemctl disable chat-system-monitor.timer
```

2. **Run setup again:**

```
```

```
sudo ./chat-system.sh setup-autostart
```

3. **Verify new daemon:**

```
sudo systemctl status chat-system
sudo tail -20 /var/log/chat-system-daemon.log
```

Summary

The continuous monitoring daemon provides:

- ✅ Real-time health monitoring (30s intervals)
- ✅ Automatic service recovery
- ✅ Intelligent restart/recreate logic
- ✅ Cooldown protection against restart loops
- ✅ Comprehensive logging
- ✅ systemd integration for reliability

This ensures your chat system stays running with minimal manual intervention!